

LevelDB 性能测试与机制探究

一、作业目的

1. 深入理解 LSM-Tree (Log-Structured Merge-Tree) 的架构特点及其在 LevelDB 中的具体实现。
2. 掌握数据库核心参数对系统吞吐量和延迟的影响。
3. 通过实测捕捉并理解 Compaction 机制带来的“写毛刺” (Write Stalls) 现象。
4. 探究不同缓存体系 (应用层 KV Cache、leveldb 底层 Block Cache) 的交互与性能差异。

二、实验环境要求

- 操作系统: Linux (Ubuntu/CentOS 等均可)
- 编程语言: C++ (C++11 及以上)
- 核心依赖: LevelDB 代码库 (<https://github.com/google/leveldb>)、pthread 库
- 测试工具: LevelDB 自带的 db_bench, 或自行编写的 C++ 压测框架。

三、作业任务

任务一: LevelDB 核心参数调优与性能对比

背景: LevelDB 提供了丰富的 `leveldb::Options` 供开发者调优。不同的业务负载 (读密集型 vs 写密集型) 需要不同的参数组合。

要求:

1. 使用 `db_bench` 或自行编写的数据生成器 (需包含顺序写、随机写、随机读等场景), 控制单次 Value 大小为 1KB, 总数据量至少为 2GB。
2. 分别修改以下参数, 记录并对比系统的 QPS 和延迟:
 - `write_buffer_size` (MemTable 大小): 对比 4MB (默认), 16MB, 64MB 的表现。
 - `block_size` (SSTable 块大小): 对比 4KB (默认), 16KB, 64KB 的表现。
 - `filter_policy` (布隆过滤器): 对比开启 Bloom Filter (10 bits/key) 与完全关闭时的随机读性能差异。
3. **产出:** 绘制柱状图或折线图, 并在报告中解释参数变化导致性能升降的原因。

任务二：长尾延迟测量与 Compaction 现象观察

背景：LSM-Tree 在后台进行 Compaction 时，会消耗大量的磁盘 I/O 和 CPU 资源，甚至阻塞前端写入，导致严重的尾延迟（Tail Latency）上升。

要求：

1. 编写一段 C++ 压测程序（或使用 db_bench 的 --histogram=1 功能），向 LevelDB 持续进行高速随机写入。
2. 强制要求数据集大小必须大于系统可用物理内存的 2 倍，以确保数据真实落盘并触发频繁的 Compaction。
3. 统计并输出写入延迟的 P50、P90、P99 和 P99.9 分位值，如果有条件可绘制随时间变化的写延迟变化图。
4. 在报告中解释为什么会出现这些写入性能波动。

任务三：应用层 KV Cache 与底层 Block Cache 的设计与对比

背景：LevelDB 内部提供了基于 LRU 的 Block Cache（缓存解压后的数据块）。在实际工程中，我们常常还会在 leveldb 之上再封装一层应用级的 KV Cache（键值对为粒度的缓存，如使用 C++ std::unordered_map 加读写锁实现）。

要求：

1. **实现 KV Cache：**在 C++ 代码中实现一个具有容量上限的简单 LRU KV Cache，作为 LevelDB 的前置代理。（即读取时先查 KV Cache，未命中再查 LevelDB）。
2. **控制变量：**
 - **情景 A（纯 Block Cache）：**关闭你的 KV Cache，开启 LevelDB 的 options.block_cache。
 - **情景 B（纯 KV Cache）：**开启你的 KV Cache，彻底关闭 LevelDB 的内部缓存（options.block_cache = leveldb::NewLRUCache(0); 以及 read_opts.fill_cache = false;）。
3. **负载测试：**使用符合 Zipf 分布（模拟二八定律热点数据）的查询请求，分别对比上述两种情景在以下操作中的 QPS 表现：
 - 高频随机读（Point Get）
 - 范围查询（Range Scan / Iterator）
 - 读写混合负载（例如 80%读+20%写，你也可以自定义比例）
4. **产出：**对比测试数据，在报告中重点论述：
 - 点查（Get）场景下，KV Cache 和 Block Cache 哪个效率更高？为什么？
 - 范围查询（Scan）场景下，你的 KV Cache 和 Block Cache 表现如何？为什么？

注意屏蔽 Page Cache 的干扰：为了测出真实的磁盘与缓存性能差异，你必须排除 Linux 操作系统 Page Cache 的影响。测试性能时，可以在运行 `leveldb` 时定期执行以下命令清空系统页缓存：`sudo sh -c 'echo 3 > /proc/sys/vm/drop_caches'`

在 `leveldb_test.cpp` 提供了一个使用 zipf 分布查询测试 `leveldb` 的示例程序，可以借鉴实现上述作业任务，在 linux 上的编译命令为 `g++ -o leveldb_demo leveldb_test.cpp -lleveldb -lpthread`

四、提交规范

1. 源代码：包含所有测试脚本、C++ 压测程序及自定义 KV Cache 的源码，需附带 Makefile 或 CMakeLists.txt，确保可一键编译运行。
2. 作业报告：
 - 包含详细的图表（吞吐量对比图、延迟对比图等）。
 - 针对各个任务的现象进行深入的理论分析，切忌只贴数据不写结论。
 - 记录实验过程中遇到的坑（例如内存泄漏、Page Cache 导致数据不准等）及解决方案。

本课程不反对使用 AI 辅助分析，但**严格禁止使用 AI 工具直接生成作业代码和报告**。

人工智能使用情况说明：请在实验报告的最后附 AI 工具的使用情况。如使用，请提交与 AI 工具的交互记录（如对话记录）；如未提交该说明，则视为本次作业的完成没有使用任何 AI 工具。请同学们诚实报告，如果出现自我声明未使用 AI，但实际使用了相关工具的情况，将严肃处理。